

Trabajo Práctico 1:

Problema de la mochila



Teoría de Algoritmos - Curso Echeverría
75.29/95.06/TB024

Integrantes:

- Finazzi Franco (113513)
- Florio Miceli Augusto Martin (113528)
- Larocca Gigirey Lucas Adrian (114218)
- Superno Lautaro (114524)

Índice:

Índice:.....	2
Introducción:.....	3
1. Fuerza bruta vs Backtracking.....	4
1.1 Supuestos.....	4
1.2 a. Pseudocódigo Fuerza Bruta y Backtracking.....	4
b. Estructuras de datos utilizadas.....	5
1.3 Seguimientos.....	6
1.4 Complejidad.....	7
1.5 Set de datos.....	7
1.6 Tiempos de ejecución.....	8
1.7 Informe de Resultados.....	9
2. Greedy.....	10
2.1 Supuestos.....	10
2.2 a. Pseudocódigo Greedy.....	10
b. Estructuras de datos utilizadas.....	11
2.3 Seguimientos.....	11
2.4 Complejidad.....	12
2.5 Set de datos.....	12
2.6 Tiempo de ejecución.....	12
2.7 Informe de Resultados.....	13
3. Programación dinámica.....	14
3.1 Supuestos.....	14
3.2 Diseño.....	14
3.2.1 Planteo tradicional: maximizar el beneficio para un peso fijo W	14
3.2.2 Planteo alternativo: minimizar el peso con un beneficio fijo B	15
3.2.3 Seguimientos.....	17
3.3 Complejidad.....	18
3.4 Sets de Datos.....	19
3.5 Tiempos de ejecución.....	20
3.6 Informe de resultados.....	21
4. Programación lineal.....	23
4.1 Supuestos.....	23
4.2 a. Ecuación fundamental y pseudocódigo.....	23
b. Estructuras de datos utilizadas.....	24
4.3 Complejidad.....	24
4.4 Set de datos.....	25
4.5 Tiempos de ejecución:.....	25
4.6 Informe de los resultados.....	25
Conclusiones.....	26

Introducción:

El problema de la mochila consiste en que, dada una mochila con una capacidad W y una cantidad de N elementos, cada uno con un peso W_i y un valor V_i , se debe determinar qué combinación de objetos guardar de forma tal que el valor total de los objetos incluidos sea máximo sin superar la capacidad límite.

En este informe se abordará este problema implementando diferentes estrategias, incluyendo fuerza bruta, backtracking, greedy, programación dinámica y programación lineal. Cada implementación contará con un análisis que profundizará en sus características y complejidad, además de proporcionar los resultados prácticos de ejecución de cada una. Al final, se presentará una comparación general entre todos los enfoques evaluando sus rendimientos y las soluciones obtenidas para cada uno.

1. Fuerza bruta vs Backtracking

1.1 Supuestos

Para ambos algoritmos los supuestos aplican iguales y son:

- La capacidad de la mochila es un valor entero positivo ($\text{capacidad} > 0$). Si no, no tendría sentido evaluarlo.
- Los pesos de los ítems son valores enteros positivos ($\text{peso} > 0$) y tampoco pueden ser negativos.
- Los beneficios de los ítems son valores enteros positivos ($\text{beneficio} > 0$).
- La cantidad de ítems es al menos 1 ($n \geq 1$). Similar a la capacidad.
- El problema es de tipo 0/1: cada ítem puede incluirse como máximo una vez, no se permiten fracciones.

1.2 a. Pseudocódigo Fuerza Bruta y Backtracking

FUERZA_BRUTA(capacidad, pesos, beneficios):

```
n ← longitud(pesos)
mejor_beneficio ← 0
mejor_mascara ← 0
```

```
para mascara desde 0 hasta  $2^n - 1$ :
```

```
    peso_total ← 0
    beneficio_total ← 0
```

```
    para i desde 0 hasta n-1:
```

```
        si el bit i de mascara está activo:
```

```
            peso_total ← peso_total + pesos[i]
            beneficio_total ← beneficio_total + beneficios[i]
```

```
    si peso_total ≤ capacidad y beneficio_total > mejor_beneficio:
```

```
        mejor_beneficio ← beneficio_total
        mejor_mascara ← mascara
```

```
retornar mejor_beneficio, mejor_mascara
```

BT(idx, peso_acum, benef_acum, mejor, capacidad, pesos, beneficios):

n ← longitud(pesos)

si idx = n:

 si benef_acum > mejor[0]:

 mejor[0] ← benef_acum

 retornar

si benef_acum + suma(beneficios[idx..n]) ≤ mejor[0]:

 retornar

si peso_acum + pesos[idx] ≤ capacidad:

 BT(idx+1, peso_acum+pesos[idx], benef_acum+beneficios[idx], mejor, capacidad, pesos, beneficios)

BT(idx+1, peso_acum, benef_acum, mejor, capacidad, pesos, beneficios)

MOCHILA_BACKTRACKING(capacidad, pesos, beneficios):

mejor ← [0]

BT(0, 0, 0, mejor, capacidad, pesos, beneficios)

retornar mejor[0]

b. Estructuras de datos utilizadas

- Arreglo de pesos (pesos): arreglo de enteros de longitud n, donde pesos[i] es el peso del ítem i.
- Arreglo de beneficios (beneficios): arreglo de enteros de longitud n, donde beneficios[i] es el beneficio del ítem i.
- Enteros auxiliares (mejor_beneficio, peso_total, beneficio_total): variables escalares para acumular y comparar valores durante la exploración.
- mejor (solo en backtracking): lista de un elemento que actúa como variable compartida entre llamadas recursivas.

1.3 Seguimientos

Ejemplo de seguimiento con un set de datos reducido para Backtracking.

Máscara	Binario	Items incluidos	Peso total	Beneficio total	¿Es viable?
0	000	0	0	0	si
1	001	1	2	3	si
2	010	1	3	4	si
3	011	2	5	7	si, y es la mejor
4	100	1	4	5	si
5	101	2	6	8	no
6	110	2	7	9	no
7	111	3	9	12	no

Ejemplo de seguimiento con un set de datos reducido para Backtracking.

idx	Acción	peso_acum	benef_acum	mejor[0]	Motivo
0	Inicio	0	0	0	—
1	Incluir ítem 0	2	3	0	$\text{peso } 2 \leq 5$
2	Incluir ítem 1	5	7	0	$\text{peso } 5 \leq 5$
3	✗ Poda cap: incluir ítem 2	—	—	7	$\text{peso } 9 > 5$
3	No incluir ítem 2 (hoja)	5	7	7	actualizo mejor
2	No incluir ítem 1	2	3	7	continúo
3	✗ Incluir ítem 2	6	8	7	$\text{peso } 6 > 5$
3	No incluir ítem 2	2	3	7	$\text{benef } 3 < 7$
1	No incluir ítem 0	0	0	7	continúo

2	Incluir ítem 1	3	4	7	continúo
3	X Incluir ítem 2	7	9	7	peso 7 > 5
3	No incluir ítem 2	3	4	7	benef 4 < 7
2	X Poda cota: no incluir ítem 1	—	—	7	cota 0+5=5 ≤ 7

1.4 Complejidad

Para ambos algoritmos la complejidad es $O(2^n n)$. Para BT en el peor caso es lo mismo ya que iría por todas las ramas. Caso contrario, corta antes.

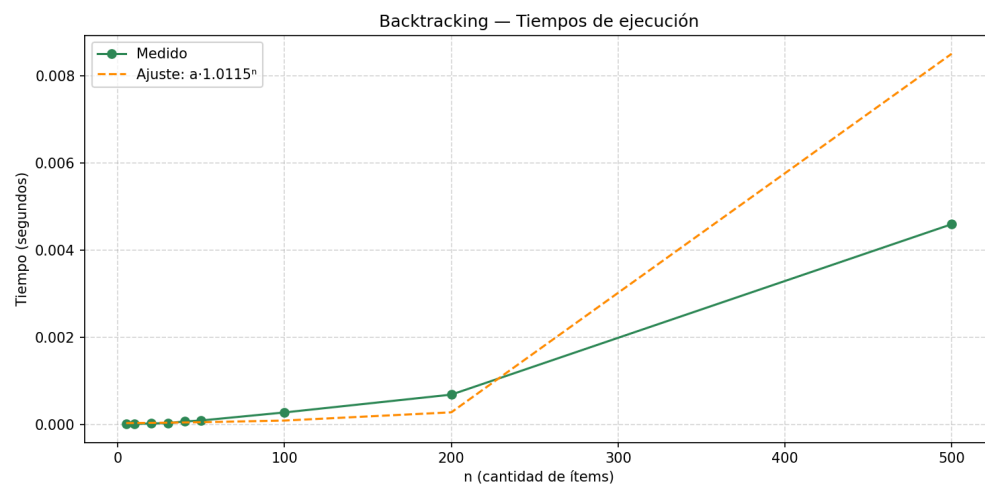
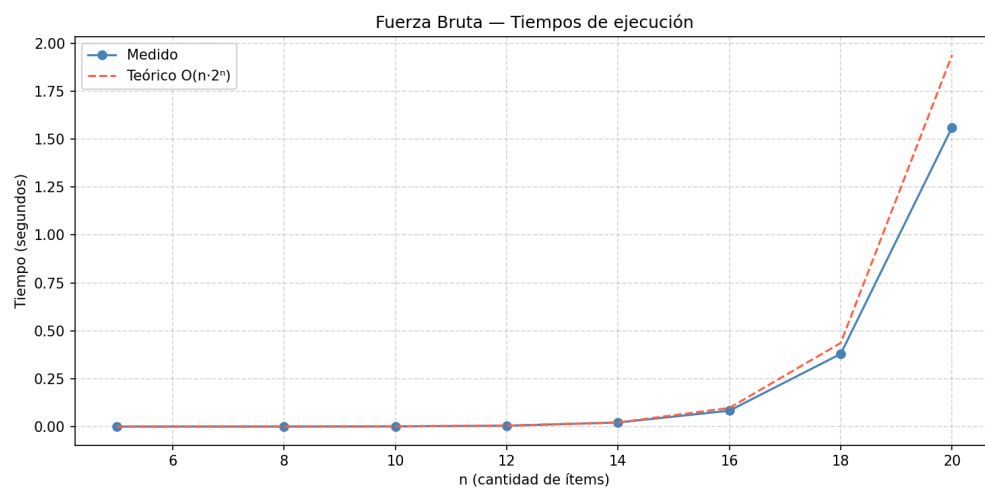
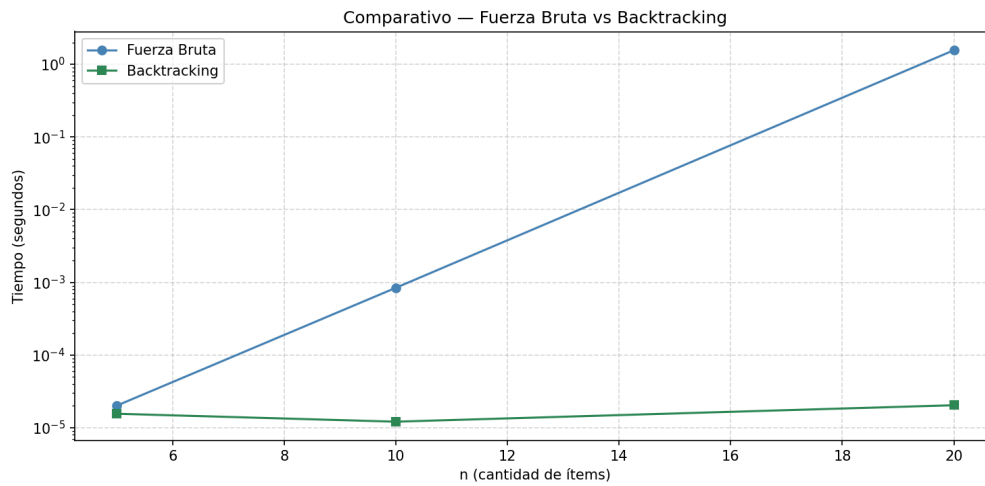
1.5 Set de datos

Se generaron instancias con crear_mochila.py para los siguientes tamaños:

- Fuerza Bruta: $n = 5, 8, 10, 12, 14, 16, 18, 20$. Se limitó a $n=20$ dado que para valores mayores el tiempo de ejecución se vuelve impráctico ($n=22$ supera los 6 segundos).
- Backtracking: $n = 5, 10, 20, 30, 40, 50, 100, 200, 500$. Se eligió un rango más amplio para diferenciar que el algoritmo escala significativamente mejor que fuerza bruta.

En ambos casos la capacidad es $n \times 50$ y los pesos y beneficios son aleatorios en los rangos definidos por crear_mochila.py.

1.6 Tiempos de ejecución



1.7 Informe de Resultados

Los resultados obtenidos se corresponden con la complejidad teórica determinada previamente. En el caso de fuerza bruta, se observa un crecimiento exponencial en los tiempos de ejecución a medida que aumenta n , ajustándose a la curva teórica $O(n \cdot 2^n)$. Para $n=20$ el tiempo supera 1.4 segundos, lo que evidencia la inviabilidad del algoritmo para instancias de mayor tamaño.

El algoritmo de backtracking, si bien tiene la misma complejidad teórica en el peor caso, demuestra en la práctica un rendimiento notablemente superior gracias a las podas por capacidad y por cota superior. Para $n=500$ el tiempo de ejecución es de apenas 0.005 segundos, mientras que fuerza bruta no puede resolver instancias de ese tamaño en tiempo razonable. La curva de ajuste del backtracking muestra una base efectiva de 1.0115, significativamente menor que 2, confirmando el comportamiento subexponencial en la práctica.

En conclusión, ambos algoritmos garantizan la solución óptima, pero el backtracking lo hace en una fracción del tiempo gracias a la poda del espacio de búsqueda.

2. Greedy

2.1 Supuestos

Para el algoritmo de greedy, tenemos los siguientes supuestos:

- La capacidad de la mochila debe ser un valor entero positivo
- Los pesos y beneficios de los elementos deben ser enteros y positivos
- Los elementos no pueden ser divisibles, (problema de tipo 0/1)
- Los elementos deben ser al menos 1

2.2 a. Pseudocódigo Greedy

Algoritmo Lectura_y_Ordenamiento(nombre_archivo):

```
Abrir nombre_archivo en modo lectura
capacidad <- Leer primera línea como entero
elementos <- Lista vacía
Para cada línea restante en el archivo:
    (peso, beneficio) <- Extraer valores de la línea
    Agregar (peso, beneficio) a la lista elementos
elementos_ordenados <- Ordenar elementos según (beneficio / peso) descendente
Retornar capacidad, elementos_ordenados
```

Algoritmo Mochila_Greedy(capacidad, elementos_ordenados):

```
peso_actual <- 0
beneficio_total <- 0

Para cada (peso, beneficio) en elementos_ordenados:
    Si peso_actual + peso <= capacidad entonces:
        peso_actual <- peso_actual + peso
        beneficio_total <- beneficio_total + beneficio

mejor_elemento_individual <- 0

Para cada (peso, beneficio) en elementos_ordenados:
    Si peso <= capacidad entonces:
        Si beneficio > mejor_elemento_individual entonces:
            mejor_elemento_individual <- beneficio
```

```
Retornar Máximo(beneficio_total, mejor_elemento_individual)
```

b. Estructuras de datos utilizadas

- Listas: Las usamos para almacenar la secuencia de elementos ordenados según su relación beneficio/peso
- Tuplas: Las usamos para representar los atributos de cada elemento (peso, beneficio)

2.3 Seguimientos

Ejemplo de seguimiento con capacidad = 10

Elem	Peso	Beneficio	B/P	Acción	Peso acum	Beneficio acum	¿Entra?
3	4	40	10	Evaluar elem	4	40	Si: 4 ≤ 10
1	7	42	6	Evaluar elem	4	40	No: 4 + 7 > 10
4	5	25	5	Evaluar elem	9	65	Si: 4 + 5 ≤ 10
2	3	12	4	Evaluar elem	9	65	No: 9 + 3 > 10

Verificación de garantía ½

Elem	Peso	Beneficio	¿Entra?	Mejor elem
3	4	40	Si: 4 ≤ 10	40
1	7	42	Si 7 ≤ 10	42
4	5	25	Si 5 ≤ 10	42
2	3	12	Si 3 ≤ 10	42

Resultado final: $\max(\text{Beneficio acumulado}, \text{mejor elemento individual}) = \max(65, 42) = 65$

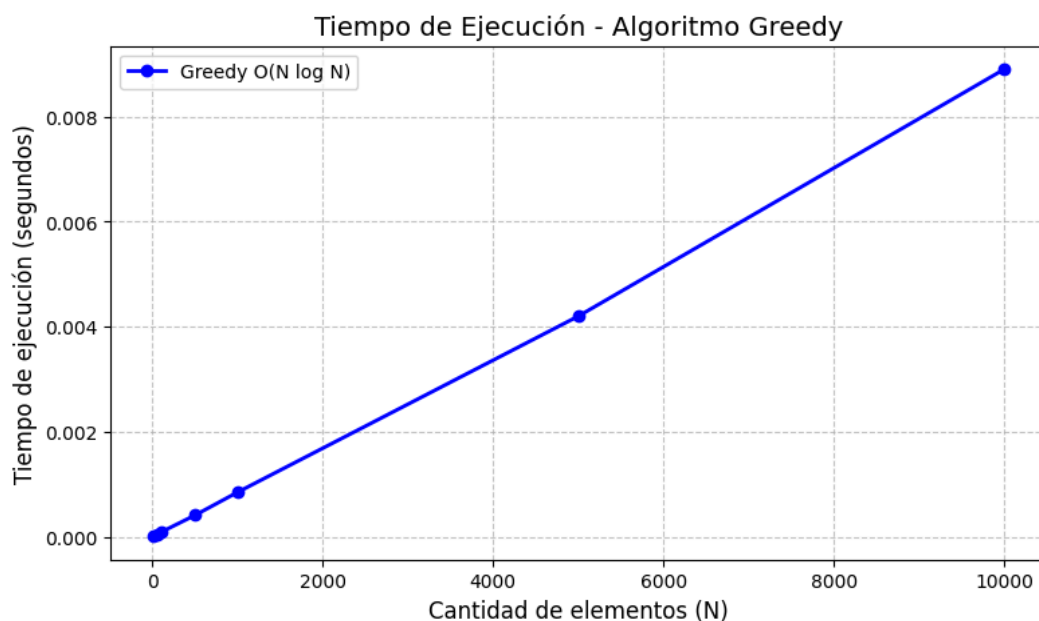
2.4 Complejidad

La complejidad de este algoritmo es $O(N \log(N))$ está dada por el ordenamiento, ya que todo lo demás como la búsqueda de mejor elemento su complejidad es $O(N)$

2.5 Set de datos

Para evaluar el desempeño y la escalabilidad del algoritmo, se construyó un conjunto de pruebas, con $n = \{10, 50, 100, 500, 1000, 5000, 10000, 50000, 100000\}$

2.6 Tiempo de ejecución



2.7 Informe de Resultados

Los resultados muestran que el algoritmo Greedy es sumamente rápido y eficiente. Al probarlo con diferentes cantidades de elementos, el tiempo de ejecución casi no aumentó, logrando procesar un conjunto de 10.000 ítems en menos de 0,01 segundos. Esto coincide perfectamente con la complejidad teórica esperada $O(N \log(N))$, donde el mayor esfuerzo de procesamiento se va en ordenar los elementos según la relación beneficio/peso.

Por el lado de los resultados, si bien esta estrategia no siempre encuentra la combinación perfecta de elementos, la regla de garantía de 1/2 asegura que el beneficio obtenido sea siempre razonable y competitivo. En resumen, es un algoritmo ideal cuando se busca una respuesta inmediata y de muy buena calidad sin complicar el procesamiento.

3. Programación dinámica

3.1 Supuestos

- Pesos, beneficios, capacidad (W) y beneficio total (B) son todos enteros positivos. En los sets utilizados el peso va de 1 a 200 y el beneficio de 1 a 1000.
- Los elementos se representan como una lista de tuplas (peso, beneficio)
- Si hay varias soluciones óptimas, la reconstrucción devuelve solo una.
- Limitación: La tabla/matriz se guarda entera en memoria para poder realizar la reconstrucción por lo tanto es acotado el tamaño de W y B que se puede utilizar.

3.2 Diseño

3.2.1 Planteo tradicional: maximizar el beneficio para un peso fijo W

-Ecuación de recurrencia:

$M[i][w]$ es el beneficio máximo que se logra usando los primeros i elementos con una capacidad w . El elemento i tiene peso p_i y beneficio v_i .

$$M[i][w] = \begin{cases} M[i-1][w] & \text{si } p_i > w \\ \max(M[i-1][w], v_i + M[i-1][w - p_i]) & \text{si } p_i \leq w \end{cases}$$

(Se adjunta una imagen de la ecuación hecha en formato LaTeX para mayor comprensión de los símbolos)

```
def mochila_max_beneficio(elementos, W):  
  
    n = len(elementos)  
  
    matriz = [[0 for _ in range(n + 1)] for _ in range(W + 1)]  
  
    for i in range(1, n + 1):  
        peso_actual, beneficio_actual = elementos[i-1]  
  
        for w in range(W + 1):  
            if peso_actual > w:  
                matriz[w][i] = matriz[w][i-1]  
            else:  
                no_incluir = matriz[w][i-1]  
                incluir = beneficio_actual + matriz[w - peso_actual][i-1]  
                matriz[w][i] = max(no_incluir, incluir)  
  
    lista_elementos = []  
    w_restante = W  
    for i in range(n, 0, -1):  
        if matriz[w_restante][i] != matriz[w_restante][i-1]:  
            elemento_incluido = elementos[i-1]  
            lista_elementos.append(elemento_incluido)  
            w_restante -= elemento_incluido[0]  
  
    return matriz[W][n], lista_elementos[::-1]
```

3.2.2 Planteo alternativo: minimizar el peso con un beneficio fijo B

-Ecuación de recurrencia:

$P[i][b]$ es el peso mínimo para conseguir un beneficio exactamente igual a b usando solo los primeros i elementos.

$$P[i][b] = \begin{cases} P[i-1][b] & \text{si } v_i > b \\ \min(P[i-1][b], p_i + P[i-1][b - v_i]) & \text{si } v_i \leq b \end{cases}$$

(Se adjunta una imagen de la ecuación hecha en formato LaTeX para mayor comprensión de los símbolos)

```
def mochila_min_peso(elementos, B):  
  
    n = len(elementos)  
    N = float('inf')  
  
    matriz = [[N for _ in range(B + 1)] for _ in range(n + 1)]  
    for i in range(n + 1):  
        matriz[i][0] = 0  
  
    for i in range(1, n + 1):  
        peso_actual, beneficio_actual = elementos[i-1]  
  
        for b in range(1, B + 1):  
            if beneficio_actual > b:  
                matriz[i][b] = matriz[i-1][b]  
            else:  
                no_incluir = matriz[i-1][b]  
                incluir = peso_actual + matriz[i-1][b - beneficio_actual]  
                matriz[i][b] = min(no_incluir, incluir)  
        if matriz[n][B] == N:  
            return None  
  
    lista_elementos = []  
    b_restante = B  
    for i in range(n, 0, -1):  
        if matriz[i][b_restante] != matriz[i-1][b_restante]:  
            elemento_incluido = elementos[i-1]  
            lista_elementos.append(elemento_incluido)  
            b_restante -= elemento_incluido[1]  
  
    return matriz[n][B], lista_elementos[::-1]
```

Referencia	Estructura de datos	Uso
Elementos	Lista de tuplas (peso, beneficio)	Almacenamiento de los datos leídos del archivo
M / P	Lista de listas	Tabla de subproblemas
N	float('inf')	Marca el beneficio inalcanzable en P. Es mayor que cualquier peso

3.2.3 Seguimientos

Ejemplo de seguimiento para un set de datos de 4 elementos dentro de la lista de tuplas

Elementos:

i	Peso	Beneficio
1	2	3
2	3	4
3	4	5
4	5	6

Planteo tradicional ($W = 5$)

Tabla $M[i][w]$, con w de 0 a 5:

i \ w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

Ejemplo: $M[2][5] = \max(M[1][5], 4 + M[1][2]) = \max(3, 4+3) = 7$. El beneficio óptimo termina siendo $M[4][5] = 7$

Reconstrucción, desde $w = 5$.

1. $i = 4$: $M[4][5] = M[3][5] = 7$, no se incluye
2. $i = 3$: $M[3][5] = M[2][5] = 7$, no se incluye
3. $i = 2$: $M[2][5] = 7 \neq M[1][5] = 3$, se incluye el (3, 4) y w vale $5 - 3 = 2$
4. $i = 1$: $M[1][2] = 3 \neq M[0][2] = 0$, se incluye el (2,3) y w vale 0

Planteo Alternativo (B = 7)

Tabla $P[i][b]$, con b de 0 a 7:

i \ b	0	1	2	3	4	5	6	7
0	0	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	2	∞	∞	∞	∞
2	0	∞	∞	2	3	∞	∞	5
3	0	∞	∞	2	3	4	∞	5
4	0	∞	∞	2	3	4	5	5

Ejemplo: $P[2][7] = \min(P[1][7], 3 + P[1][3]) = \min(\infty, 3+2) = 5$. El peso mínimo para el beneficio fijo 7 termina siendo $P[4][7] = 5$.

Reconstrucción empezando en $b = 7$.

1. $i = 4$: $P[4][7] = P[3][7] = 5$, no se incluye.
2. $i = 3$: $P[3][7] = P[2][7] = 5$, no se incluye.
3. $i = 2$: $P[2][7] = 5 \neq P[1][7] = \infty$, se incluye (3, 4) y b pasa a $7 - 4 = 3$
4. $i = 1$: $P[1][3] = 2 \neq P[0][3] = \infty$, se incluye (2,3) y b pasa a 0.

3.3 Complejidad

Ambos algoritmos poseen la misma estructura por lo que el análisis es igual para los dos. El algoritmo con el planteo tradicional tiene una complejidad de $\Theta(n*W)$ y el alternativo $\Theta(n*B)$.

Planteo tradicional:

- Crear e inicializar la tabla de $(n+1) * (W+1)$ celdas cuesta $\Theta(n*W)$.
- El doble ciclo for de n iteraciones externas por W internas con comparaciones, operaciones aritméticas y máximo, sucede todo en tiempo constante $\Theta(n*W)$.
- Reconstrucción: n pasos de costo $\Theta(n)$
- Total: **$\Theta(n*W)$**

Planteo alternativo:

- Crear e inicializar la tabla de $(n+1) * (B+1)$ celdas, y la columna 0 en cero cuesta $\Theta(n*B)$.
- El doble ciclo for de n por B iteraciones te costo constante: $\Theta(n*B)$.
- Reconstrucción: $\Theta(n)$.
- Total: **$\Theta(n*B)$**

3.4 Sets de Datos

Los sets de datos fueron generados con el código provisto por la cátedra crear_mochila.py como se solicita en la consigna de este TP.

Los tamaños de n utilizados para las pruebas son $n = \{20, 40, 80, 120, 160, 200, 250\}$

Los sets de datos utilizados fueron elegidos para poder representar el comportamiento de cada algoritmo desde un caso pequeño y más sencillo hasta uno mucho más grande como puede ser el de $n = 250$, pero sin llegar a un exceso de uso de memoria.

El número de n posible para realizar la evaluación del comportamiento de ambos algoritmos es acotado dado que la tabla se guarda en memoria para luego realizar la reconstrucción. Esto genera que para valores de n muy grandes la capacidad de almacenamiento deba ser cada vez mayor.

3.5 Tiempos de ejecución

Gráfico del algoritmo tradicional.

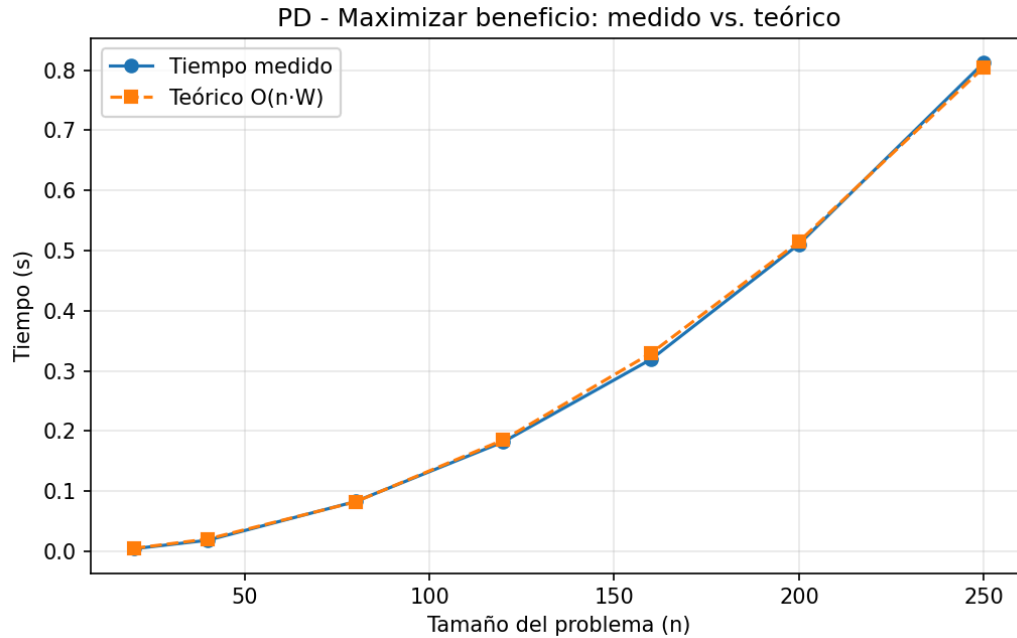


Gráfico del algoritmo Alternativo

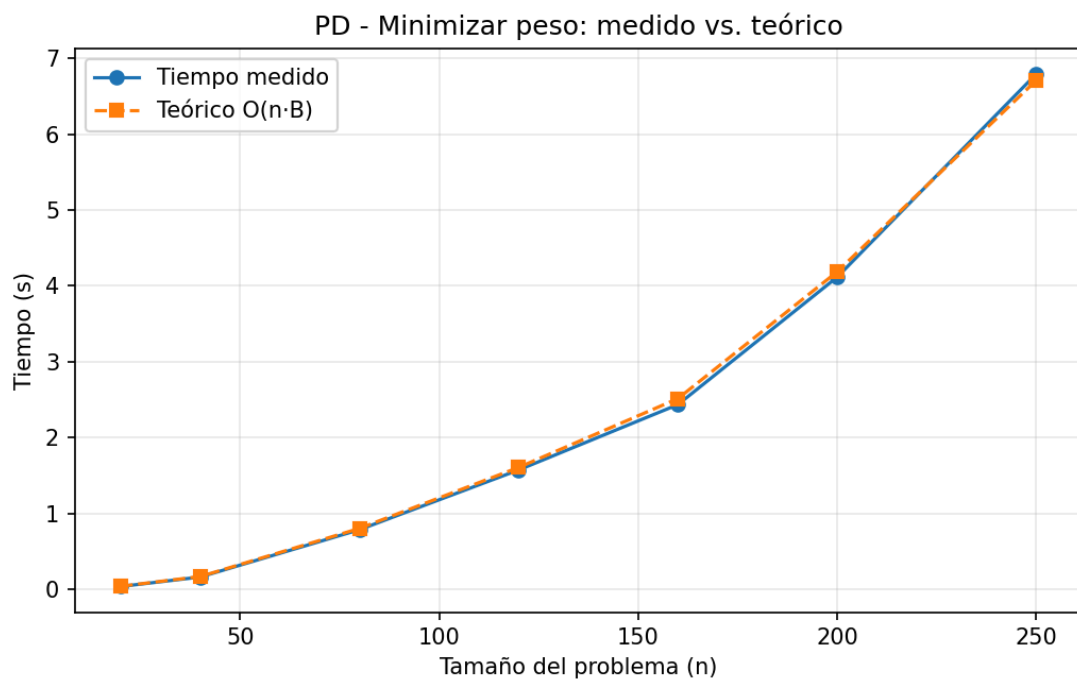
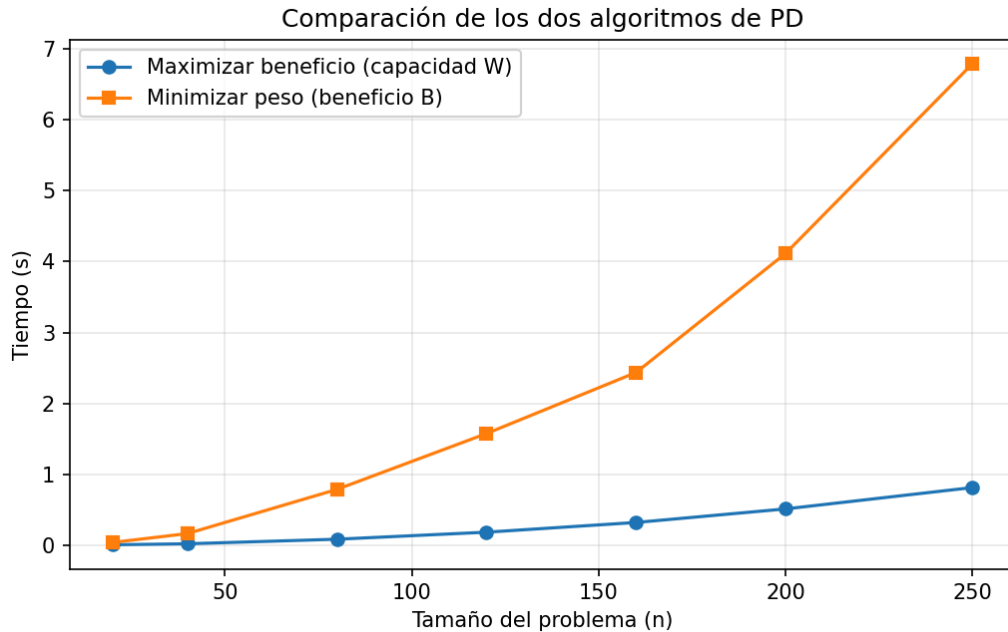


Gráfico comparativo de los dos planteos:



3.6 Informe de resultados

A partir de las mediciones realizadas con el set de datos elegido podemos observar que para ambos planteos, tanto el tradicional como el alternativo, las curvas dadas a partir de los valores utilizados para la medición se corresponden claramente con la curva teórica de su complejidad.

Las mayores diferencias las podemos encontrar en la comparación entre los dos planteos. Si bien tienen la misma forma de complejidad, $\Theta(n \cdot W)$ para el tradicional y $\Theta(n \cdot B)$ para el alternativo, el algoritmo del planteo alternativo posee un tiempo de ejecución mayor de aproximadamente B/W veces el tradicional. Esto se condice con lo esperado: La cátedra proporcionó, en el algoritmo de generación de la mochila, valores de beneficio entre 1 y 1000 y de peso entre 1 y 200, con una capacidad $W = n \cdot 50$. Como el beneficio objetivo B se tomó como el beneficio óptimo del planteo tradicional, este resulta mucho mayor que W . Por lo tanto, aunque el costo por celda para cada algoritmo sea el mismo, el ancho de la tabla para el planteo alternativo será mucho más amplio respecto al tradicional y tiene más celdas para completar.

Ejemplo de la variación B/W entre los tiempos de cada algoritmo.

$n = 250$, $W=12500$, $B=100906 \rightarrow 100906/12500 \approx 8,072$

siendo que el tradicional tardó $t_1 = 0,8124$ y el alternativo $t_2 = 6,7941$

$\rightarrow 0,8124 * 8,072 \approx 6,558$. El número no es exacto debido a costos en el llenado de las celdas y almacenamiento de las tablas, pero es una aproximación y relación válida que conviene destacar al haber utilizado el beneficio óptimo del algoritmo tradicional como el beneficio objetivo del algoritmo alternativo.

4. Programación lineal

4.1 Supuestos

Para resolver el problema mediante programación lineal se tienen en cuenta los siguientes supuestos:

- Los elementos de la mochila o están o no están, no se permiten fracciones
- Tanto los valores peso w_i y valor v_i de los elementos tienen que ser positivos, además del tamaño total W de la mochila
- La resolución del problema se realizará utilizando la biblioteca Pulp

4.2 a. Ecuación fundamental y pseudocódigo

Para poder plantear el problema mediante programación lineal y resolverlo primero tenemos que armar la función objetivo y las restricciones, en el caso del problema de la mochila se verían de la siguiente manera:

-Variable: la única variable que vamos a utilizar es para saber si un ítem está o no en la mochila, esto lo podemos representar como una variable binaria x_i (1 si llevo el elemento, 0 si no lo llevo):

$$x \in \{0, 1\}$$

-Ecuación fundamental: partiendo de la variable ahora podemos definir la ecuación fundamental, que en este caso es de maximización ya que lo que queremos obtener es el mayor beneficio posible entre los elementos capaces de entrar en nuestra mochila sin superar la capacidad de la misma:

$$Z(max) = \sum_{i=1}^n v_i x_i$$

-Restricciones: en este caso la restricción va a estar delimitada por la suma total del peso de todos los elementos dentro de la mochila y hacer que estos no sobrepasen el peso máximo:

$$\sum_{i=1}^n w_i x_i \leq W$$

Con las ecuaciones ya planteadas podemos escribir el pseudocódigo:

```
mochilaProgramacionLineal(items, capacidadMochila):
    n = longitud(items)
    problema = CrearProblema(maximización, "Mochila")
    x[] (variables)
    beneficio[] (beneficio de los ítems)
    peso[] (peso de los ítems)

    for i in range n:
        variable = crearVariable(Item_n, variableBinaria)
        agregar variable a x

        beneficio_item = item[i] beneficio
        agregar (beneficio_item * variable) a beneficio

        peso_item = item[i] peso
        agregar (peso_item * variable) a peso
    end for

    problema.objetivo(suma(beneficios))
    problema.restricción(suma(pesos) <= capacidadMochila)

    problema.resolver
    return resultado
```

b. Estructuras de datos utilizadas

- Listas: las usamos para tener la lista total de ítems posibles
- Tuplas: Las usamos para representar los atributos de cada ítem (peso, beneficio)

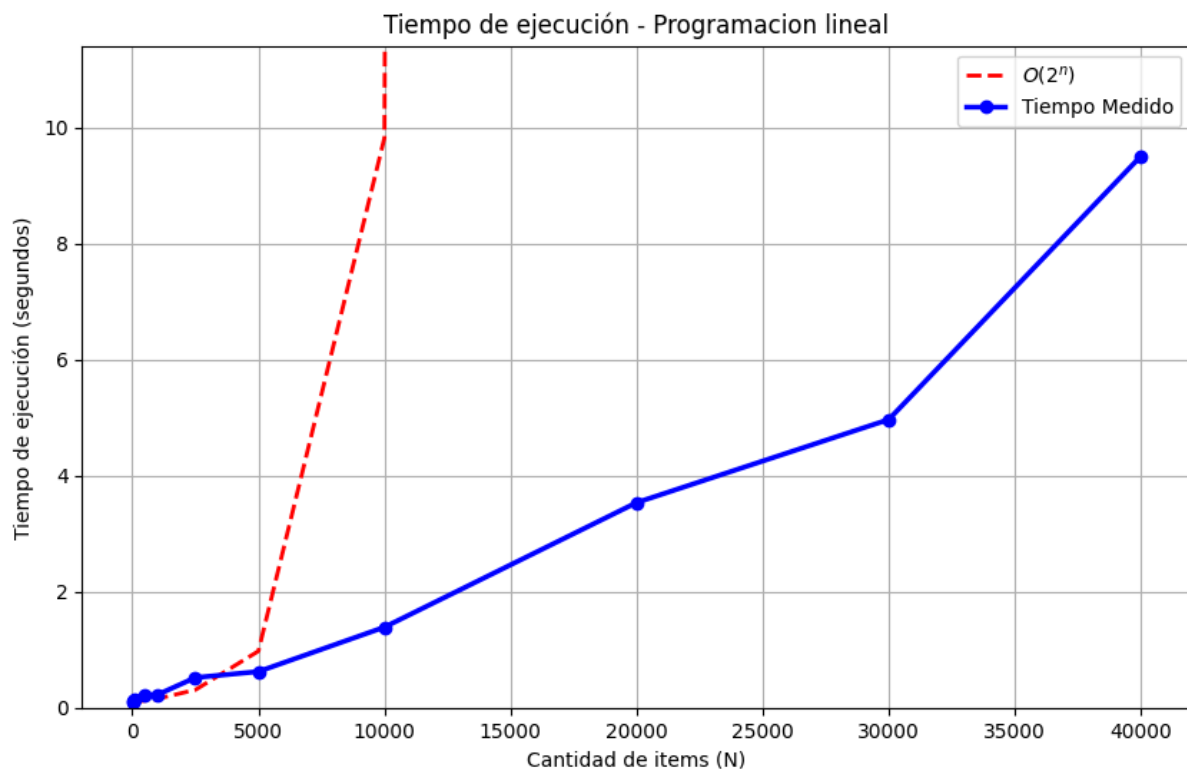
4.3 Complejidad

La complejidad del algoritmo está dictada principalmente por la operación `problema.solve()`, esta misma es de $O(2^n)$ ya que viene de la forma en la que pulp resuelve problemas de programación lineal que es con branch and bound

4.4 Set de datos

Para este caso se eligió un set de datos que incluye los siguientes valores (cada valor significa la cantidad de ítems generados aleatoriamente), el tamaño de la mochila es $n * 50$:
[10, 50, 100, 500, 1000, 2500, 5000, 10000, 20000, 30000, 40000]

4.5 Tiempos de ejecución:



Los tiempos obtenidos fueron $\approx$
[0.09, 0.12, 0.14, 0.22, 0.22, 0.51, 0.62, 1.38, 3.53, 4.96, 9.50]

4.6 Informe de los resultados

Con el gráfico se evidencia que la curva teórica está alejada de los resultados que se obtienen en la práctica. La complejidad teórica $O(2^n)$ solo ocurre en los peores de los casos donde el algoritmo de solución se ve forzado a analizar todas las combinaciones posibles del problema. Que en realidad esto ocurra es muy poco probable, ya que a medida que se va resolviendo el problema internamente se van eliminando las combinaciones no viables dejándonos con tiempos más razonables para poder obtener soluciones óptimas.

Conclusiones

A lo largo del informe pudimos analizar cinco enfoques y las cualidades de cada uno. Uno de los enfoques más fáciles de implementar y con buenos tiempos de ejecución fue greedy, aun así, su principal desventaja es que no nos garantiza la solución óptima sino una buena. Otros enfoques como fuerza bruta proporcionan una implementación sencilla pero que al incrementar la cantidad de elementos a evaluar se vuelven inviables debido al tiempo que tardan en ejecutarse. Para mejorar esto se puede recurrir al backtracking que, si bien mantiene la misma complejidad teórica, en la práctica se realiza en tiempos mucho menores.

Por otro lado, mediante la Programación Dinámica podemos obtener la solución óptima de una forma consistente y predecible, dividiendo el problema general en subproblemas. Finalmente la programación lineal nos ofrece un planteo diferente a los demás, si podemos identificar de manera correcta las variables y restricciones necesarias vamos a obtener una solución eficiente y sencilla de implementar que, a pesar de lo que aparenta con su complejidad, nos permite resolver el problema de forma eficiente.

Para concluir podemos decir que la forma en la que decidimos resolver un algoritmo es muy importante. Cada uno tiene sus propias características que los diferencian y dependiendo de lo que queramos obtener podemos inclinarnos hacia uno u otro (tiempo, simplicidad, exactitud de la solución, etc..).